Lab 1: Flip array
Geavanceerde Computerarchitectuur

Sien Peerlinck & Thibaut Dalez

February 23, 2024

1 Introduction

The goal of this lab was learning the basics of GPU programming and Cuda. This was done by creating a program that flips an array. In this report the structure of the program will be explained.

2 Code

In this section, we will explain the working of the code step by step. The entire program can be found in appendix A.

The code begins on lines 18-19 by defining the input array and output array, both of size N (which is defined on line 4). Line 21 defines pointers to the addresses of the above arrays, so we can directly address them later on.

On lines 29 and 30 we allocate two places in the memory with the size of the arrays and we let the address pointers point to them. On line 33 we then put the input array in the provided memoryspace on the GPU memory.

The blocksize is predefined, this is on line 36. Then the gridsize is defined (line 37), this calculates the number of threads that are needed to cover the array. If the grid =(N/blocksize), and N is not divisible by blocksize, not enough threads would be allocated because the integer function always rounds down. By adding the blocksize - 1 to the function we overcome this problem.

Now we call the CPU kernel to actually flip the array. The working of the GPU kernel is explained below. We call the GPU kernel and pass the blocksize and gridsize, then we pass the needed variables for the arrayflipfunction (the addresspointers to both arrays and the arraysize N).

On line 44 we take the output array and copy it from the GPU memory to the CPU memory. On lines 47-48, the allocated memoryplaces (for input- and output array) are freed. This always has to be done because otherwise your memoryspace will run out.

Lines 51-61 are used to print the input array and the output array to visually check them for correctness. The output for an arraysize of 8 looks like in figure 1. When not enough threads were created, the output array ended in trailing zeros. For example, when only one block of 4 threads was created, the output looked like this: 7 6 5 4 0 0 0 0.

The function in the GPU kernel (line 9-15) requires as input a pointer to the input array, ouput array and the size. The actual flipping operation happens on line 13. Each thread takes an element from the end of the input array and writes it to the position in the output array. [size - tid - 1] ensures the right element is taken. Tid is different for each thread en depends on the blok that is currently being serviced and the number of the thread.

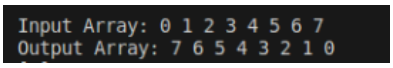


Figure 1: Output for arraysize N=8.

A Program

```cpp
#include <iostream>
#include <cuda_runtime.h>

#define N 1000 // Size of the arrays

// GPU kernel
__global__ void arrayFlip(int *input, int *output, int size) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;  // total index = numberr of block we're in now x total number of block + number of thread

    if (tid < size) {  // stops after going through the entire array
        output[tid] = input[size - tid - 1];  // this is the line that turns the arrat
    }
}


// CPU code
int main() {
    int inputArray[N];  // define arrays of size N
    int outputArray[N];

    int *d_inputArray, *d_outputArray;  // define pointer to address of above

    // Fill input array with numbers
    for (int i = 0; i < N; i++) {
        inputArray[i] = i;
    }

    // Allocate memory on GPU on the address pointed to by the defined pointer with the size of N times the size of an int (so the total size of the array)
    cudaMalloc((void **)&d_inputArray, N * sizeof(int));
    cudaMalloc((void **)&d_outputArray, N * sizeof(int));

    // Put defined input array in the memory allocated above
    cudaMemcpy(d_inputArray, inputArray, N * sizeof(int), cudaMemcpyHostToDevice); // cpu memory to GPU memory

    // Define grid and block size
    int blockSize = 256;  // number of threads in block
    int gridSize = (N + blockSize - 1) / blockSize;  // number of blocks in grid, made large enough to cover all N without having too much threads (resource waste) (rrounds down)
    // in case of N=2000 and block_size = 256 : gridsize=8.8 => 8 ; 8*256=2048>2000

    // Launch GPU kernel to flip the array
    arrayFlip<<<gridSize, blockSize>>>(d_inputArray, d_outputArray, N);  // geef pointers naar arrays mee

    // put calculated output array in allocated memory
    cudaMemcpy(outputArray, d_outputArray, N * sizeof(int), cudaMemcpyDeviceToHost);  // gpu memory to CPU memory

    // Free memory on GPU (because if you allocate memory, you always have to set it free again)
    cudaFree(d_inputArray);
    cudaFree(d_outputArray);

    // Print the result
    std::cout << "Input Array: ";
    for (int i = 0; i < N; i++) {
        std::cout << inputArray[i] << " ";
    }
    std::cout << std::endl;

    std::cout << "Output Array: ";
    for (int i = 0; i < N; i++) {
        std::cout << outputArray[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

Listing 1: ArrayFlip